

D4 — Trainee Guide

Python for AI Scaffolding · 26-Jun-2026 · upload-to-VM + every exercise explained

FORGE D2 — Start Here

Session: Python for AI Scaffolding · 26-Jun-2026 · 4 hrs **Programme:** FORGE — Forward Deployed Engineer Academy (Batch 1)

Welcome. Today you build the **type-safe Python AI scaffold** that every later session reuses (29-Jun DDD, 02-Jul integration, 06-Jul Azure migration). This kit is everything you need.

What's in this kit

File / folder	What it is
<code>00_START_HERE.md</code>	This file — read it first.
<code>01_UPLOAD_TO_YOUR_VM.md</code>	How to get this kit onto your Azure VM and open it in VS Code. Do this first.
<code>02_EXERCISE_GUIDE.md</code>	The exercise itself — what you'll learn, what to do, how to check each step.
<code>CHEAT_SHEET.md</code>	Copy-adapt code snippets for the four patterns.
<code>lab/starter/</code>	Your working folder. A near-complete app with 3 gaps (TODOs) for you to fill.
<code>lab/solution/</code>	The complete reference. Open this only after you've attempted the TODOs — see the note below.

The 60-second version

1. **Get it on the VM** → follow `01_UPLOAD_TO_YOUR_VM.md`.
 2. **Open `lab/starter/` in VS Code** and run `./setup.sh` (creates the `.venv`, runs the tests).
 3. **See red tests** — that's intentional. Each red test names the gap you must fill.
 4. **Fill the 3 TODOs** in order: `app/models.py` → `app/client.py` → `app/function_calling.py`. Full instructions in `02_EXERCISE_GUIDE.md`.
 5. **Tests go green** → start the API → open `/docs` in the browser.
 6. **Raise your hand** when everything is green.
-

About the lab/solution/ folder

The reference solution is included so you can check your work and keep it after the session.

Try every TODO yourself first. Copying the solution skips the muscle you're here to build — the test failures are the lesson. Treat solution/ as the answer key you open *after* you've had a real go, or when you're truly stuck and the cheat sheet didn't unblock you.

What "done" looks like today

- pytest is green in lab/starter/ (20 passed, 1 skipped).
- The API starts and `http://127.0.0.1:8000/docs` shows a typed POST /complete.
- Sending a wrong-typed input to /docs returns a clean 422 (not a crash).
- You can say, in one sentence, **why** a strict boundary + retries keeps an AI app alive in production.

01 • Open the Kit in VS Code on Your Azure VM

You do **all of today's work inside your Techademy Azure VM** — never on your own laptop. **VS Code is already installed inside the VM**, and that's the editor you'll use for this session and every coming session. There is **nothing to install on your laptop**. Budget ~10 minutes; do it before the session starts.

You need: access to your **Techademy Azure VM** (follow the **Azure access guide** your trainer shared — it tells you exactly how to log in), and the kit file `D02_python_ai_scaffolding_Trainee_Kit.zip` (from the LMS / the link your trainer shared). If you can't reach your VM, tell your trainer **before** the session.

Step 1 — Log into your VM and open VS Code

1. **Log into your Azure VM** following your **Azure access guide** (your trainer provides the VM address and your sign-in). You'll land on the VM's desktop — this is your working machine for the programme.
2. **Open VS Code inside the VM** (it's pre-installed — look for the VS Code icon on the desktop/taskbar, or run code in the VM's terminal). Everything from here happens **on the VM**, not your laptop.

You do **not** install VS Code, and you do **not** use "Remote-SSH" — you're already working *inside* the VM, so its local VS Code is all you need.

Step 2 — Get the kit onto the VM

Option A — download it inside the VM (simplest): open a browser **in the VM** and download `D02_python_ai_scaffolding_Trainee_Kit.zip` from the LMS / the link your trainer shared. It lands in the VM's Downloads folder.

Option B — download via the VM terminal: open VS Code's terminal (**Terminal → New Terminal**) and run:

```
wget "<KIT_DOWNLOAD_URL>" -O D02_python_ai_scaffolding_Trainee_Kit.zip # or:
curl -L "<URL>" -o D02_python_ai_scaffolding_Trainee_Kit.zip
```

Your trainer may have **pre-placed the kit on the VM** already — if you see a `D02_python_ai_scaffolding_Trainee_Kit.zip` (or an unzipped trainee-kit/) in your home folder, skip the download.

Then unzip it in the VM terminal:

```
unzip D02_python_ai_scaffolding_Trainee_Kit.zip
```

You now have a trainee-kit/ folder on the VM.

Step 3 — Open the lab and create its environment

1. In the VM's VS Code: **File** → **Open Folder...** → navigate into **trainee-kit/lab/starter** → **OK**. (Open the *lab* folder, not the kit root — VS Code's Python tools work per-folder.)
2. Open a terminal (**Terminal** → **New Terminal**) and run:
`./setup.sh`

This creates a local **.venv**, installs dependencies, and runs the tests. You'll see some **red (failing) tests** — that is expected; they mark the gaps you'll fill today.

3. **Select the interpreter:** Cmd/Ctrl+Shift+P → **Python: Select Interpreter** → choose the one ending in **./venv/bin/python**. (VS Code usually offers it automatically once **.venv** exists.)
 - If **Python: Select Interpreter** isn't available, install the **Python** extension (publisher: Microsoft) from the Extensions panel (Cmd/Ctrl+Shift+X) **inside the VM's VS Code** — it's usually already there. That's the only extension you need.

You're ready. Go to **02_EXERCISE_GUIDE.md**.

Troubleshooting

Symptom	Fix
Can't reach / log into the VM	Re-check the Azure access guide ; confirm the VM is started. Ask your trainer if it's still unreachable.
<code>./setup.sh</code> : Permission denied	Run <code>chmod +x setup.sh</code> then <code>./setup.sh</code> again.
<code>python3.12</code> : command not found	Your VM image is missing 3.12 — tell your trainer; <code>setup.sh</code> also works if <code>uv</code> is installed.
<code>unzip</code> : command not found	<code>sudo apt-get update && sudo apt-get install -y unzip</code> (Ubuntu VMs).
No Python: Select Interpreter command	Install the Microsoft Python extension in the VM's VS Code (Extensions panel).
Tests all pass immediately (no red)	You opened <code>lab/solution/</code> by mistake. Open lab/starter/ instead.

02 • Exercise Guide — Python for AI Scaffolding

Work in `lab/starter/`. By the end you'll have a type-safe AI service with a strict input/output contract, a resilient retry client, and a typed function-calling harness — all tested.

The scenario (why this exercise exists)

You're an FDE who shipped an LLM feature. It worked in **staging**. In **production** the same model provider returns the *same information in a different JSON shape*, and your code crashes in front of the client. The model didn't break — **your contract assumption broke**.

Stage	What the provider returns
staging	<code>{"text": "...", "model": "...", "finish_reason": "stop"}</code>
prod (drifted)	<code>{"message": {"content": "..."}, "model": "...", "stop_reason": "stop"}</code>

Today you build the boundary that makes this a *caught, typed event* instead of a 2 a.m. incident. Treat the model like a **flaky external vendor**: type it, validate it, retry it, time it out.

How the app is laid out

File	Role	You touch it?
<code>app/models.py</code>	<code>CompletionRequest</code> (strict in) + <code>CompletionResponse</code> (validated out)	✅ TODO 1
<code>app/client.py</code>	<code>LLMClient</code> — wraps a provider, retries, validates the boundary	✅ TODO 2
<code>app/function_calling.py</code>	<code>call_tool()</code> — parse + validate the model's tool-call args	✅ TODO 3
<code>app/errors.py</code>	<code>TransientError</code> , <code>PermanentError</code> , <code>TypedError</code>	read only
<code>app/providers/mock.py</code>	offline, deterministic provider; can emit prod drift + fake 429s	read only
<code>app/config.py</code>	<code>get_provider()</code> — picks mock/openai/azure from an env var	read only
<code>app/main.py</code>	FastAPI app — <code>/healthz</code> , <code>POST /complete</code>	read only
<code>tests/</code>	the suite that tells you what to build	run it constantly

The **tests are your spec**. Run them after every change. Each failing test names the exact thing to fix.

Workflow for every TODO

1. Read the failing test(s) named in the TODO — they define "correct."
2. Open the file, find the # TODO(...) comment.
3. Implement it (use CHEAT_SHEET.md for the pattern — adapt, don't paste blindly).
4. Re-run the acceptance command. Green → move on. Stuck → peek at lab/solution/ for that one file.

Run tests in the VS Code terminal (interpreter = ./venv/bin/python):

```
./venv/bin/python -m pytest -q          # whole suite
./venv/bin/python -m pytest tests/test_models.py -q # one file
```

Or use the **Testing** beaker icon in VS Code's sidebar to run/debug tests by clicking.

Part 0 — See the crash (5 min, no code)

What you'll learn: what "works on my machine" failure actually looks like.

What to do: your trainer runs the untyped version live; you watch `raw["text"]` throw `KeyError: 'text'` the moment the prod shape arrives. Notice: no warning, no type, just a crash. Hold that picture — everything below is the fix.

Part 1 — TODO 1 · Strict contracts + drift normalization (app/models.py)

What you'll learn: Pydantic v2 as a *guardrail at the boundary* — strict input typing, and a "before" validator that absorbs payload drift so the rest of the app sees one clean shape.

What to do: in `CompletionResponse`, complete the `normalize` validator so it maps **both** shapes to `{text, model, finish_reason}`:

- staging shape already matches → pass it through unchanged.
- prod shape has a message dict → pull `message.content` into `text`, and use `stop_reason` as `finish_reason`.

Pattern is in CHEAT_SHEET.md §1. Key idea: `@model_validator(mode="before")` runs *before* field assignment, so it's where you rename/reshape keys. `strict=True` on the request means `prompt=123` is rejected, not silently cast.

How to verify:

```
./venv/bin/python -m pytest tests/test_models.py tests/test_contract.py -q
```

Green when `test_response_normalizes_prod_drift` passes. (The contract test is the **handoff artefact** later sessions rely on — it pins this boundary.)

Part 2 — TODO 2 · Resilient retry client (`app/client.py`)

What you'll learn: wrapping a flaky provider in a typed client with **exponential backoff** and a clean error boundary — the difference between "retried and recovered" and "crashed."

What to do: implement `LLMClient.complete()`:

1. Loop up to `self.max_retries + 1` attempts.
2. `raw = await self.provider.complete(request)`, then **validate**: return `CompletionResponse.model_validate(raw)`.
3. On `TransientError` (e.g. a 429): if attempts remain, await `asyncio.sleep(self.base_delay * 2 ** attempt)` and continue.
4. When retries are exhausted, raise `PermanentError(...)`.

Pattern: `CHEAT_SHEET.md §2`. Note the two failure classes already defined in `errors.py` — **transient = retry, permanent = give up**. That distinction is the whole point.

How to verify:

```
./venv/bin/python -m pytest tests/test_resilient_client.py tests/test_api.py -q
```

Green when the client validates prod drift, retries-then-succeeds, and gives up after max retries.

Part 3 — TODO 3 · Typed function-calling (`app/function_calling.py`)

What you'll learn: function/tool calling is a **contract negotiation** — the model hands you a JSON *string* of arguments and sometimes breaks the contract. Your job: parse, validate, and return a **typed error** instead of crashing.

What to do: inside `call_tool()`'s try block:

- `raw["arguments"]` is a JSON **string** → `json.loads(...)` it first.
- Validate the dict into `tool_model` via `tool_model.model_validate(...)`.
- On `json.JSONDecodeError` **or** `ValidationError`, return `ToolCallResult(ok=False, error=TypedError(code="bad_tool_args", message=str(exc), retryable=False))`.

- On success, return `ToolCallResult(ok=True, args=parsed.model_dump())`.

Pattern: CHEAT_SHEET.md §3. The outer `TransientError` retry loop is already written for you — you're only filling the parse/validate step.

How to verify:

```
./venv/bin/python -m pytest tests/test_function_calling.py -q
```

Part 4 — Run the service and break it on purpose (10 min)

What you'll learn: how the strict boundary shows up as a clean HTTP 422, and how to read /docs.

What to do:

```
./venv/bin/python -m uvicorn app.main:app --reload --port 8000
```

(or press **F5** / Run "Run API (uvicorn)"). Then open <http://127.0.0.1:8000/docs>:

- POST /complete with {"prompt": "Hello!", "max_tokens": 50} → 200 with a text field.
 - POST /complete with {"prompt": 123, "max_tokens": 50} (integer prompt) → 422 with the exact field error. That 422 is strict Pydantic doing its job — a *feature*, not a bug.
-

Finish line

```
./venv/bin/python -m pytest -q      # expect: 20 passed, 1 skipped
```

The 1 skip is the live-key smoke test (`test_real_providers_optional`) — skipped because there's no real API key in your VM. That's fine.

Commit your work:

```
git add app/models.py app/client.py app/function_calling.py
git commit -m "d2: implement 3 TODOs – normalize, retry, function-calling"
```

Reflection (write one line): *What single pattern from today will you carry into the capstone?*
Then raise your hand. You've built the scaffold every later session stands on.

Cheat Sheet — D2 · Python for AI Scaffolding

Quick-reference snippets for the four teaching points. Paste and adapt — do not copy blindly.

1 — Pydantic v2 strict model + cross-field validator

```
from pydantic import BaseModel, model_validator

class CompletionRequest(BaseModel):
    model_config = {"strict": True}  # rejects type coercion (int prompt → 422, not silent cast)

    prompt: str
    max_tokens: int = 512

class CompletionResponse(BaseModel):
    text: str
    model: str
    finish_reason: str

    @model_validator(mode="before")
    @classmethod
    def normalize(cls, values: dict) -> dict:
        """Absorb staging-vs-prod payload drift at the boundary."""
        if "message" in values:
            # prod drifted shape: {"message": {"content": "..."}, "stop_reason": "..."}
            values["text"] = values["message"]["content"]
            values["finish_reason"] = values.pop("stop_reason", "stop")
            del values["message"]
            # staging shape already matches – pass through unchanged
        return values
```

Key: mode="before" runs before field assignment, so you can rename keys. strict=True means prompt=123 raises ValidationError, not a silent coercion to "123".

2 — Retry / exponential backoff (async)

```
import asyncio
from app.errors import TransientError, PermanentError
```

```

async def complete_with_retry(
    provider,
    request,
    max_retries: int = 3,
    base_delay: float = 0.5,
):
    last_exc: Exception | None = None
    for attempt in range(max_retries + 1):
        try:
            return await provider.complete(request)
        except TransientError as exc:
            last_exc = exc
            if attempt < max_retries:
                await asyncio.sleep(base_delay * 2**attempt) # 0.5s, 1s, 2s,
4s ...
            raise PermanentError(f"Failed after {max_retries} retries: {last_exc}")

```

Key: base * 2**attempt gives 0.5 → 1 → 2 → 4 s. Add random.uniform(0, 0.1) * base for jitter in production. The official OpenAI SDK already retries 429/5xx internally (max_retries param) — wrap that, do not reinvent it.

3 — Function-calling: parse JSON args → validate into Pydantic model

```

import json
from pydantic import BaseModel, ValidationError
from app.errors import TypedError # <- Lab/starter/app/errors.py
from app.models import ToolCallResult # <- Lab/starter/app/models.py

class WeatherArgs(BaseModel):
    location: str
    unit: str = "celsius"

def parse_tool_args(raw: dict, tool_model: type[BaseModel]) -> ToolCallResult:
    """raw["arguments"] is a JSON *string* from the model – parse then validate."""
    try:
        parsed_args = json.loads(raw["arguments"]) # step 1: JSON string → dict
        parsed = tool_model.model_validate(parsed_args) # step 2: dict → typed model
        return ToolCallResult(ok=True, args=parsed.model_dump())
    except (json.JSONDecodeError, ValidationError) as exc:
        return ToolCallResult(
            ok=False,
            error=TypedError(code="bad_tool_args", message=str(exc), retryabl

```

```
e=False),  
    )
```

Key: The model returns arguments as a JSON-encoded *string*, not a dict. Always `json.loads()` first; then validate. Return a typed error — never let a `JSONDecodeError` propagate as an unhandled exception to the caller.

4 — FastAPI with Pydantic request + response models

```
from fastapi import FastAPI, HTTPException  
from app.models import CompletionRequest, CompletionResponse  
from app.client import LLMClient  
from app.errors import PermanentError  
  
app = FastAPI(title="AI Scaffold")  
client = LLMClient()  
  
@app.post("/complete", response_model=CompletionResponse)  
async def complete(request: CompletionRequest) -> CompletionResponse:  
    """  
    response_model=CompletionResponse:  
    - serializes the return value through the model (strips extra fields)  
    - FastAPI generates the typed OpenAPI schema automatically  
    """  
    try:  
        return await client.complete(request)  
    except PermanentError as exc:  
        raise HTTPException(status_code=503, detail=str(exc))
```

Key: `response_model` tells FastAPI what shape to advertise in `/docs` and what to validate on the way out. A 422 from bad input is Pydantic strict mode working correctly — show it to learners as a feature.

Footnote — Providers are swappable

Azure OpenAI is the programme's deployment target (migrated 06-Jul). Treat OpenAI / Anthropic / Azure OpenAI as **swappable adapters behind the interface, not the destination** — no SDK import belongs in business logic, so you can re-point the provider with a one-line change. When you need an exact model ID, copy it from the provider's current docs — never type it from memory (they change without notice).